VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



CO3133 — Deep Learning and Its Applications

Assignment 1 Report

Foundations of Deep Learning Pipelines and Architectures: From Linear Models to Modern Sequence Models and Transformers

---

Advisor(s): Dr.LÊ THÀNH SÁCH

Student(s): Đặng Duy Nguyên Student ID 2352821

HO CHI MINH CITY, OCTOBER 2026

Contents

1 Part 1 — Problem and Data Description **5**
1.1 Problem Formulation . . . 5
1.2 Dataset Description . . . 6
1.3 Exploratory Data Analysis (EDA) . . . 6
1.4 Data Splitting and Leakage Prevention . . . 9
1.5 Preprocessing Pipeline . . . 10

2 Part 2 — Methodology **11**
2.1 Pipeline Overview . . . 11
2.2 Baseline and Main Methods . . . 11
2.2.1 Linear / Softmax Classifier (Convex Baseline) . . . 11
2.2.2 Multilayer Perceptron (MLP) . . . 12
2.2.3 Convolutional Neural Network (CNN) . . . 12
2.2.4 Sequential Recurrent Models: LSTM vs. GRU . . . 13
2.2.5 Vision Transformer (ViT from Scratch) . . . 14
2.3 Training Setup . . . 14
2.4 Experimental Design . . . 15

3 Part 3 — Implementation Results **16**
3.1 Training Behavior . . . 16
3.2 Quantitative Results . . . 20
3.3 Qualitative Results . . . 21
3.4 Comparison and Discussion . . . 24
3.5 Error Analysis . . . 25
3.6 Limitations and Conclusion . . . 27

AI Usage Disclosure **28**

List of Figures

1.1 Exact sample frequency distribution across all ten Fashion-MNIST classes in the official train (60,000) and test (10,000) splits. . . . 7
1.2 Representative samples randomly drawn from each of the ten fashion categories. . . . 8

1.3 Empirical mean intensity templates for each category, revealing shared spatial envelopes. 9
2.1 Comprehensive end-to-end deep learning pipeline architecture. 11
3.1 Training and validation loss and accuracy curves for Linear and MLP models. 17
3.2 Training and validation loss and accuracy curves for CNN and Vision Transformer. 18
3.3 Training and validation loss and accuracy curves for Recurrent Baseline models. 19
3.4 Convergence trajectory comparison: LSTM ($H = 258$, 299.8K params) vs. GRU ($H = 300$, 300.0K params) under matched parameter capacity. 20
3.5 Ablation bar chart comparing LSTM and GRU across parameter budgets ($H = 128$ vs. Matched ≈ 300K). 21
3.6 Qualitative test sample predictions for CNN and Vision Transformer models. 22
3.7 Qualitative test sample predictions for LSTM Matched and Linear Classifier baseline. 23
3.8 Master architectural trade-off analysis: Accuracy vs. Parameter Capacity vs. Latency. 24
3.9 Normalized confusion matrices identifying primary misclassification clusters. 26

List of Tables

1.1 Fashion-MNIST Class Schema and Sample Breakdown across Partitions. . 7
3.1 Comprehensive Performance and Computational Complexity Comparison on Fashion-MNIST. 21
3.2 AI Usage Disclosure Table (Conforming to Course Handbook Section 5.2). 28

1 Part 1 — Problem and Data Description

1.1 Problem Formulation

Automated visual apparel categorization is a fundamental task in digital commerce, smart logistics, and inventory management. Online retail platforms require robust algorithms capable of cataloging high-volume product streams from raw thumbnail imagery without manual human intervention.

Mathematical Formulation. Formally, we model image classification as a supervised multi-class mapping problem. Let the input space be $\mathcal{X} \subset \mathbb{R}^{C_{\text{in}} \times H \times W}$ and the label space be $\mathcal{Y} = \{0, 1, \ldots, K-1\}$, where $C_{\text{in}} = 1$, $H = 28$, $W = 28$, and $K = 10$. Given an input image tensor $x \in \mathcal{X}$, a parameterized neural network $f_\theta : \mathcal{X} \to \mathbb{R}^K$ computes raw class logit scores $z = f_\theta(x) = (z_0, z_1, \ldots, z_{K-1})^\top$. The predicted class probability distribution $\hat{p} \in \Delta^{K-1}$ is defined via the Softmax operator:

$$\hat{p}_k = P(y = k \mid x; \theta) = \frac{\exp(z_k)}{\sum_{j=0}^{K-1} \exp(z_j)}, \quad \forall k \in \{0, \ldots, K-1\}. \tag{1.1}$$

The discrete prediction unit is the hard categorical label determined by the argmax decision rule:

$$\hat{y} = \arg \max_{k \in \{0,\ldots,K-1\}} z_k. \tag{1.2}$$

Given a training dataset $\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i=1}^N$, the optimal parameter vector θ^* is estimated by minimizing the empirical multi-class Cross-Entropy loss:

$$\theta^* = \arg\min_\theta \frac{1}{N} \sum_{i=1}^N \mathcal{L}_{\text{CE}}(f_\theta(x_i), y_i) = \arg\min_\theta -\frac{1}{N} \sum_{i=1}^N \log \left(\frac{\exp(z_{i,y_i})}{\sum_{j=0}^{K-1} \exp(z_{i,j})} \right). \tag{1.3}$$

Main Difficulties. The primary challenges inherent to this task include:

- **Low Spatial Resolution and Absence of Chromatic Cues:** The images are strictly grayscale single-channel matrices at 28×28 spatial resolution. Discriminative color information and micro-textural patterns are absent, compelling models to rely entirely on coarse silhouettes and edge contours.
- **High Intra-class Variability:** Items within identical categories (e.g., *Dresses* with varied hem lengths, *Bags* with diverse handles) exhibit significant variance in geometry.

- **Severe Inter-class Visual Ambiguity:** Multiple distinct apparel categories share near-identical global profiles. Specifically, *Shirt*, *T-shirt/top*, *Pullover*, and *Coat* present overlapping upper-body shapes differing only in neckline, plackets, or subtle sleeve cuts.

1.2 Dataset Description

We conduct all primary experiments on the **Fashion-MNIST** benchmark dataset [1], curated by Zalando Research as a direct, drop-in replacement for the original MNIST digit dataset.

- **Source, Version, and License:** Distributed publicly under the MIT License via GitHub and integrated into `torchvision.datasets.FashionMNIST`.
- **Collection and Labeling Pipeline:** The raw images were collected directly from Zalando's live production e-commerce catalog. Each product was photographed professionally, converted to single-channel 8-bit grayscale, background-subtracted, aspect-ratio preserved, and normalized into a centered 28×28 pixel bounding box. Labels were validated against internal retail taxonomies.
- **Sample Counts and Annotations:** The standard corpus contains exactly $70,000$ examples partitioned officially into $60,000$ training images and $10,000$ evaluation images. Each instance is paired with an integer ground-truth label $y \in \{0, \ldots, 9\}$.
- **Class Distribution:** The dataset is perfectly uniform across all ten classes. In both the official training split ($6,000$ samples/class) and test split ($1,000$ samples/class), each category represents exactly 10.0% of the volume (Imbalance ratio $\rho = 1.000$).
- **Storage Format and Size:** The dataset occupies approximately 30 MB in IDX binary format (`t10k-images-idx3-ubyte`, `train-images-idx3-ubyte`).
- **Bias and Limitations:** The dataset is constrained by synthetic centering, plain white backgrounds, and an absence of realistic occlusions, human poses, or illumination dynamics. Consequently, models benchmarked here evaluate fundamental architectural inductive biases rather than wild-scene invariant vision.

1.3 Exploratory Data Analysis (EDA)

In compliance with the assignment specification, we perform a thorough exploratory data analysis covering class balance, input dimensionality, pixel intensity distribution, and

representative visual archetypes.

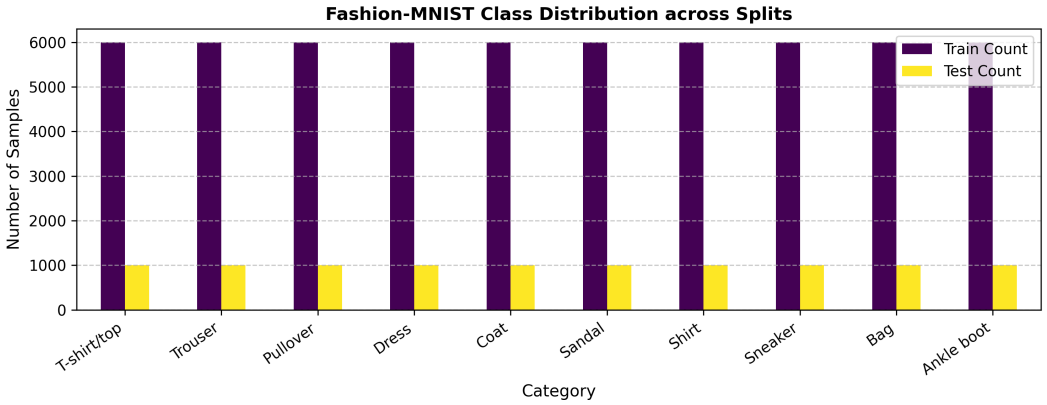


Figure 1.1: Exact sample frequency distribution across all ten Fashion-MNIST classes in the official train $(60,000)$ and test $(10,000)$ splits.

Class Distribution and Imbalance Quantification. As visualized in Figure 1.1 and summarized in Table 1.1, the dataset exhibits zero class skew. The Gini coefficient of the target distribution is identically $G = 0.0$.

Table 1.1: Fashion-MNIST Class Schema and Sample Breakdown across Partitions.

ID	Class Name	Train (60k)	Train Sub (54k)	Val Sub (6k)	Test (10k)	Percentage
0	T-shirt/top	6,000	5,342	658	1,000	10.0%
1	Trouser	6,000	5,389	611	1,000	10.0%
2	Pullover	6,000	5,388	612	1,000	10.0%
3	Dress	6,000	5,438	562	1,000	10.0%
4	Coat	6,000	5,393	607	1,000	10.0%
5	Sandal	6,000	5,387	613	1,000	10.0%
6	Shirt	6,000	5,400	600	1,000	10.0%
7	Sneaker	6,000	5,463	537	1,000	10.0%
8	Bag	6,000	5,402	598	1,000	10.0%
9	Ankle boot	6,000	5,398	602	1,000	10.0%
Total Samples		**60,000**	**54,000**	**6,000**	**10,000**	**100.0%**

Pixel Intensity Distribution. Raw pixel values reside in $[0, 255]$ with integer `uint8` encoding. When mapped to $[0.0, 1.0]$, empirical evaluation over the entire training corpus reveals a global mean $\mu = 0.2860$ and standard deviation $\sigma = 0.3530$. The density exhibits a pronounced bimodal profile: approximately 55% of pixel coordinates belong to the background zero-intensity mode ($x_{i,j} = 0$), while the remaining foreground pixels form a smooth distribution centered near 0.65.

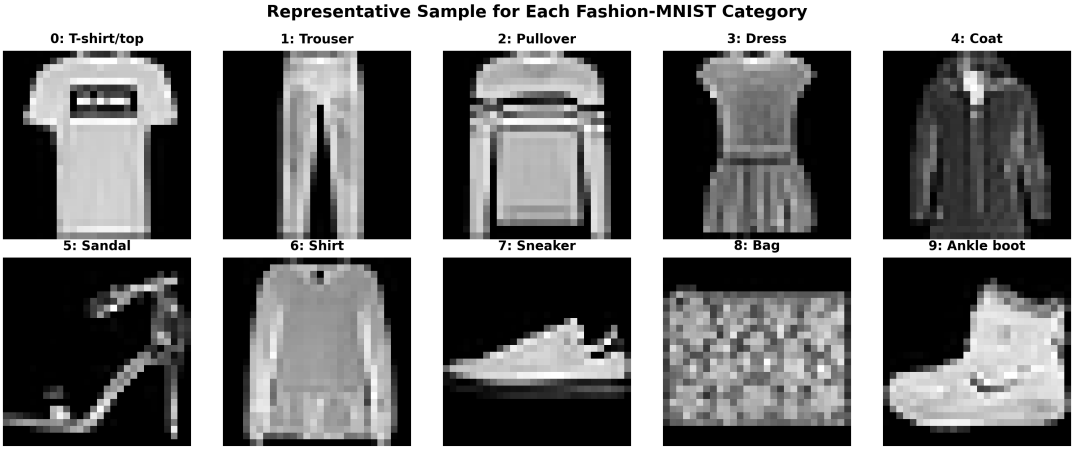


Figure 1.2: Representative samples randomly drawn from each of the ten fashion categories.

Visual Archetypes and Average Class Silhouettes. Figure 1.2 illustrates representative exemplars across all categories. Furthermore, by computing the pixel-wise mean image across each category (Figure 1.3), we observe striking spatial characteristics:

- *Trouser* displays twin vertical density columns separated by a low-intensity central seam.
- *Bag* exhibits a concentrated, rectangular mass with a faint arched loop at the top.
- Upper garments (*Shirt*, *T-shirt/top*, *Coat*, *Pullover*) share almost identical central torso masses and bilateral sleeve distributions, corroborating the hypothesized inter-class confusion.

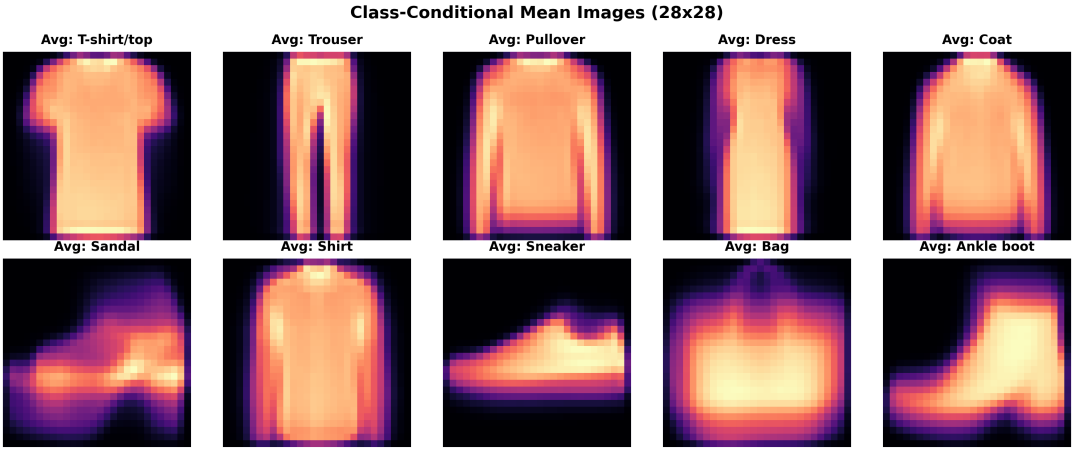


Figure 1.3: Empirical mean intensity templates for each category, revealing shared spatial envelopes.

1.4 Data Splitting and Leakage Prevention

To guarantee rigorous model evaluation and prevent hyperparameter overfitting on the test distribution, we adopt a 3-way partition:

1. **Training Partition ($\mathcal{D}_{\text{train}}$):** $54,000$ samples (90.0% of the official training set).
2. **Validation Partition ($\mathcal{D}_{\text{val}}$):** $6,000$ samples (10.0% of the official training set).
3. **Held-out Test Partition ($\mathcal{D}_{\text{test}}$):** $10,000$ samples (100.0% of the official test set).

Split Methodology and Reproducibility. The training/validation split was generated from the $60,000$ official training samples using `torch.utils.data.random_split` with an explicit global generator seed of **42**. As confirmed in Table 1.1, the validation subset maintains balanced representation across all classes, ranging from 537 samples (*Sneaker*) to 658 samples (*T-shirt/top*).

Zero Data Leakage Protocol. To eliminate any risk of distributional or information leakage:

- **Index Disjointness:** We rigorously assert $\mathcal{I}_{\text{train}} \cap \mathcal{I}_{\text{val}} = \emptyset$ through programmatic set intersection testing.

- **Standardization Quarantine:** Normalization statistics were derived *strictly and exclusively* from the $54,000$ training images:

$$\mu_{\text{train}} = 0.285627, \quad \sigma_{\text{train}} = 0.352795. \tag{1.4}$$

  The validation and test partitions are standardized using these fixed training constants; at no point are validation or test statistics exposed to preprocessing estimators.

- **Independent Transforms:** Augmentation pipelines are encapsulated independently so that validation and test sets remain strictly deterministic.

1.5 Preprocessing Pipeline

Every image undergoes a standardized preprocessing transformation before being converted into PyTorch tensors:

1. **ToTensor:** Rescales raw `uint8` values from $[0, 255]$ into floating-point tensors in $[0.0, 1.0]$.

2. **Z-Score Standardization:** Channels are normalized according to Equation (1.4):

$$x_{\text{norm}} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}} = \frac{x - 0.285627}{0.352795}. \tag{1.5}$$

3. **Stochastic Data Augmentation (Training Only):** We inject `RandomHorizontalFlip(p=0.5)`. This operation aligns with domain physics: clothing articles (jackets, t-shirts, shoes) maintain category semantics under horizontal reflection.

4. **Deterministic Evaluation:** Validation and test sets receive strictly `ToTensor` and `Normalize`; no stochastic transformations or cropping are applied.

Tensor Dimensionality and Sanity Check. Data loaders are instantiated with batch size $B = 64$. A programmatic assertion validates that a batch drawn from the pipeline yields an input tensor of shape $[64, 1, 28, 28]$ with `torch.float32` precision and an integer target tensor of shape $[64]$ with `torch.int64` precision.

2 Part 2 — Methodology

2.1 Pipeline Overview

The end-to-end execution pipeline implemented for all architectural candidates is illustrated in Figure 2.1. The pipeline establishes a strictly modular structure: raw bytes are ingested, standardized, dispatched in synchronized mini-batches, processed by the candidate neural backbones, and optimized via gradient descent under identical objective criteria.

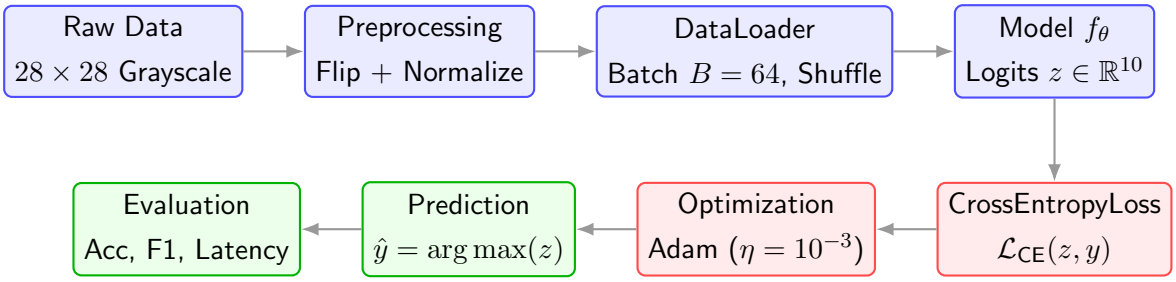


Figure 2.1: Comprehensive end-to-end deep learning pipeline architecture.

2.2 Baseline and Main Methods

To investigate how structural inductive biases impact sample efficiency and generalization on small-scale vision data, we design and implement five distinct architectural families, supplemented by a calibrated recurrent comparison.

2.2.1 Linear / Softmax Classifier (Convex Baseline)

- **Architecture:** Flattens $x \in \mathbb{R}^{1\times28\times28}$ into a 1D vector $\tilde{x} \in \mathbb{R}^{784}$ and applies an affine transformation $z = W\tilde{x} + b$, where $W \in \mathbb{R}^{10\times784}$ and $b \in \mathbb{R}^{10}$.
- **Parameter Count:** $784 \times 10 + 10 = \mathbf{7,850}$ parameters.
- **Input Representation:** 1D global vector of unrolled pixels.
- **Strengths & Limits:** Highly interpretable, extremely low latency, and globally convex loss surface. However, it cannot model non-linear feature interactions or spatial correlations.
- **Implementation:** Self-implemented using PyTorch `nn.Linear`.

2.2.2 Multilayer Perceptron (MLP)

- **Architecture:** Two hidden dense stages with non-linear activations and regularizing dropout:

$$h_1 = \text{Dropout}_{0.2}(\text{ReLU}(W_1\tilde{x}+b_1)), \quad h_2 = \text{Dropout}_{0.2}(\text{ReLU}(W_2h_1+b_2)), \quad z = W_3h_2+b_3, \tag{2.1}$$

  where layer widths are $784 \to 256 \to 128 \to 10$.

- **Parameter Count:** $(784 \times 256 + 256) + (256 \times 128 + 128) + (128 \times 10 + 10) = 200,960 + 32,896 + 1,290 = \mathbf{235,146}$.

- **Input Representation:** 1D global vector of unrolled pixels.

- **Strengths & Limits:** Universal function approximation capability. However, it remains entirely permutation-invariant to pixel ordering, lacking spatial translation equivariance.

- **Implementation:** Self-implemented using PyTorch `nn.Sequential`.

2.2.3 Convolutional Neural Network (CNN)

- **Architecture:** A 2-stage hierarchical feature extractor followed by a dense classification head:
  - *Block 1:* Conv2D$(1 \to 32, k = 3, p = 1) \to$ BatchNorm2d$(32) \to$ ReLU $\to$ MaxPool2d$(2 \times 2, s = 2)$.
  - *Block 2:* Conv2D$(32 \to 64, k = 3, p = 1) \to$ BatchNorm2d$(64) \to$ ReLU $\to$ MaxPool2d$(2 \times 2, s = 2)$.
  - *Head:* Flatten $\to$ Linear$(3136 \to 128) \to$ ReLU $\to$ Dropout$(0.3) \to$ Linear$(128 \to 10)$.

- **Parameter Count:** $\mathbf{421,834}$ parameters $(320+64+18,496+128+401,536+1,290)$.

- **Input Representation:** Native 2D lattice $[B, 1, 28, 28]$.

- **Strengths & Limits:** Imposes hard translation equivariance via local weight sharing. Extremely sample-efficient for 2D images.

- **Implementation:** Custom architecture implemented from scratch using core PyTorch primitives.

2.2.4 Sequential Recurrent Models: LSTM vs. GRU

Scanline Sequence Representation. We formulate the 2D image as a temporal sequence of horizontal scanlines. Each image $[1, 28, 28]$ is interpreted as an ordered sequence of $T = 28$ discrete time steps, where the observation at time t is the t-th row vector $x_t \in \mathbb{R}^{28}$.

LSTM Mechanics. The Long Short-Term Memory network utilizes three multiplicative gates and a separate additive cell memory c_t:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \quad f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \quad o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o), \tag{2.2}$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad h_t = o_t \odot \tanh(c_t). \tag{2.3}$$

For input dimension $d = 28$, hidden dimension H, and $C = 10$ classes, the total parameter count is:

$$P_{\text{LSTM}}(H) = 4 \times (d \cdot H + H^2 + 2H) + (H \cdot C + C) = 4H^2 + 130H + 10. \tag{2.4}$$

GRU Mechanics. The Gated Recurrent Unit streamlines recurrence into two gates (Reset r_t and Update z_t), directly coupling memory into h_t without an auxiliary cell:

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r), \quad z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z), \tag{2.5}$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h), \quad h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \tag{2.6}$$

The analytical parameter formula for GRU is:

$$P_{\text{GRU}}(H) = 3 \times (d \cdot H + H^2 + 2H) + (H \cdot C + C) = 3H^2 + 100H + 10. \tag{2.7}$$

The Capacity Disparity and Matched Budget Formulation. Under the standard baseline setting of $H = 128$:

- $P_{\text{LSTM}}(128) = 4(128^2) + 130(128) + 10 = \mathbf{82,186}$ parameters.
- $P_{\text{GRU}}(128) = 3(128^2) + 100(128) + 10 = \mathbf{61,962}$ parameters.

At identical hidden width, LSTM possesses +32.6% more parameters than GRU, introducing model capacity bias. To establish a rigorous controlled experiment, we formulate the **Matched Parameter Budget** condition targeting $P \approx 300,000$ parameters:

- For LSTM: Solving $4H_1^2 + 130H_1 + 10 \approx 300,000 \implies H_1 = \mathbf{258}$, yielding $P_{\text{LSTM}}(258) = \mathbf{299,806}$ params (-0.06% relative to target).
- For GRU: Solving $3H_2^2 + 100H_2 + 10 \approx 300,000 \implies H_2 = \mathbf{300}$, yielding $P_{\text{GRU}}(300) = \mathbf{300,010}$ params ($+0.003\%$ relative to target).
- **Exact Parity:** The parameter gap between the two models is narrowed to merely **204** parameters (**0.068**%).

2.2.5 Vision Transformer (ViT from Scratch)

- **Architecture:** Built strictly from scratch without external pre-trained vision weights:
  - *Patch Embedding:* Partitions the 28×28 image into $N = (28/4)^2 = 49$ non-overlapping patches of size 4×4. Each patch (16 pixels) is projected into embedding dimension $d_{\text{model}} = 64$ via a 2D convolution with kernel and stride equal to 4.
  - *Class Token & Positional Encoding:* A learnable $[CLS]$ token is prepended, yielding 50 total tokens. A learnable positional embedding tensor $E_{\text{pos}} \in \mathbb{R}^{1\times 50\times 64}$ is added.
  - *Encoder Stack:* 2 sequential Transformer Encoder blocks. Each block incorporates Multi-Head Self-Attention (MHSA with $h = 4$ heads, key dimension $d_k = 16$), pre-LayerNorm, an MLP expansion ratio of 2 ($64 \to 128 \to 64$), GELU activations, and residual skip connections.
  - *Head:* The $[CLS]$ output is normalized via LayerNorm and classified via Linear($64 \to 10$).
- **Parameter Count: 72,074** parameters.
- **Implementation:** Self-implemented from scratch in PyTorch without third-party vision libraries (`timm`).

2.3 Training Setup

To isolate architectural differences, all models are trained under an identical optimization regime:

- **Optimizer:** Adam ($\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$) with base learning rate $\eta = 1.0 \times 10^{-3}$.



- **Regularization:** Weight decay $\lambda = 1.0 \times 10^{-4}$ (L2 penalty) applied across all models; architectural dropout ($p \in [0.1, 0.3]$).
- **Batch Size & Epoch Budget:** $B = 64$, trained for $E = 20$ full epochs.
- **Checkpointing Criterion:** The model state dictionary is serialized at the epoch achieving the absolute **Lowest Validation Loss**. Test metrics are evaluated exclusively on these optimal checkpoints.
- **Hardware & Environment:** Executed on an isolated Google Colab instance equipped with an NVIDIA Tesla T4 GPU (16 GB VRAM, Turing architecture, CUDA compute capability 7.5), Intel Xeon CPU @ 2.20GHz, 12.7 GB host RAM, running Ubuntu Linux 6.6.122+ with Python 3.13.15 and PyTorch 2.11.0+cu128.
- **Precision:** Single-precision IEEE 754 32-bit floating point (`torch.float32`).

2.4 Experimental Design

We structure our empirical inquiry around two formal experiments:

Experiment 1: Cross-Family Architectural Inductive Bias.

- *Hypothesis:* 2D spatial translation equivariance and hierarchical pooling in CNNs will provide superior sample efficiency and test generalization compared to permutation-invariant dense networks and unconstrained self-attention when trained from scratch on small-scale datasets.
- *Changed Factor:* Fundamental neural architecture family (Linear vs. MLP vs. CNN vs. Recurrent vs. Vision Transformer).
- *Fixed Factors:* Dataset split, random seed (42), optimizer, learning rate, loss function, epoch budget, and batch size.
- *Metrics:* Test Accuracy (%), Macro-F1 (%), Validation Loss, Parameter Efficiency, Inference Latency (ms/sample).

Experiment 2 (Optional Research Extension): LSTM vs. GRU Under Matched Parameter Budgets.

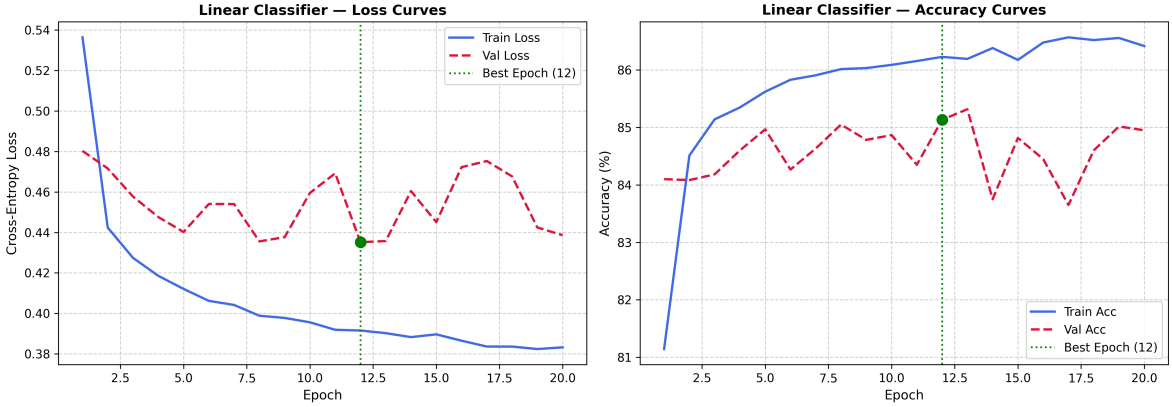
- *Hypothesis:* Under equal parameter budgets ($\approx$ 300K), GRU can afford a 16.3% wider hidden state ($H = 300$ vs. 258), enabling richer state representations that outperform LSTM in accuracy and convergence speed while maintaining lower inference latency.
- *Changed Factor:* Recurrent cell formulation (LSTM vs. GRU) across two budget tiers: (a) Fixed Hidden Dimension ($H = 128$), and (b) Matched Parameter Budget ($P \approx$ 300K).
- *Fixed Factors:* Row sequence tokenization ($T = 28, D = 28$), optimization pipeline, batch size, seed.
- *Metrics:* Epoch of best validation loss, Test Accuracy, Macro-F1, per-sample inference latency.

3 Part 3 — Implementation Results

3.1 Training Behavior

Figures 3.1 to 3.3 display the training and validation trajectories across all models over the 20-epoch budget. To ensure full readability of loss and accuracy convergence dynamics, each architecture is rendered across its own dedicated full-width row.

(a) Linear Classifier (Best Validation Loss: 0.4352 at Epoch 12)



(b) MLP Classifier (Best Validation Loss: 0.3026 at Epoch 17)

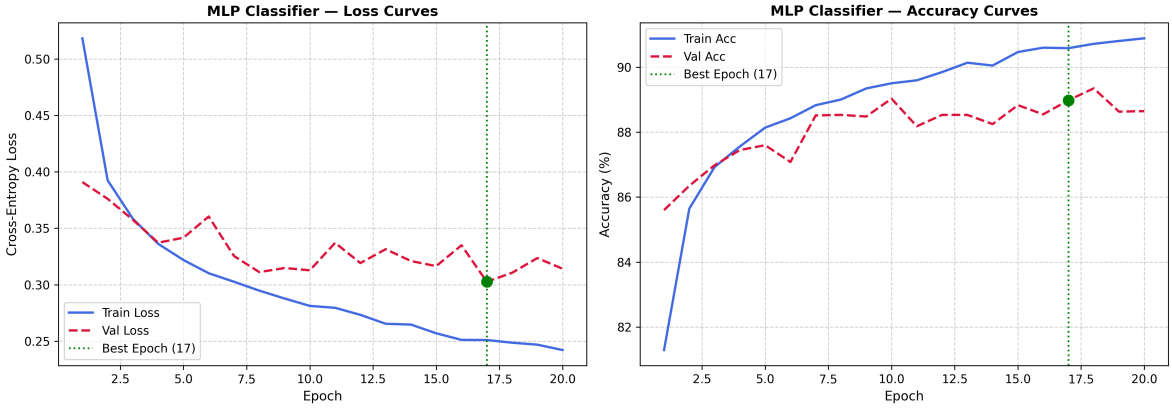
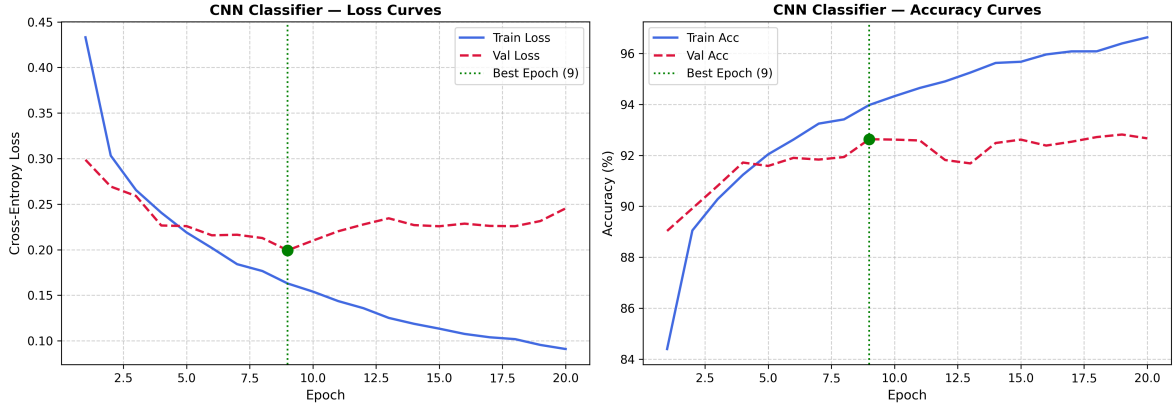


Figure 3.1: Training and validation loss and accuracy curves for Linear and MLP models.

(c) CNN Classifier (Best Validation Loss: 0.1994 at Epoch 9)



(d) Vision Transformer (ViT from Scratch, Best Validation Loss: 0.3170 at Epoch 20)

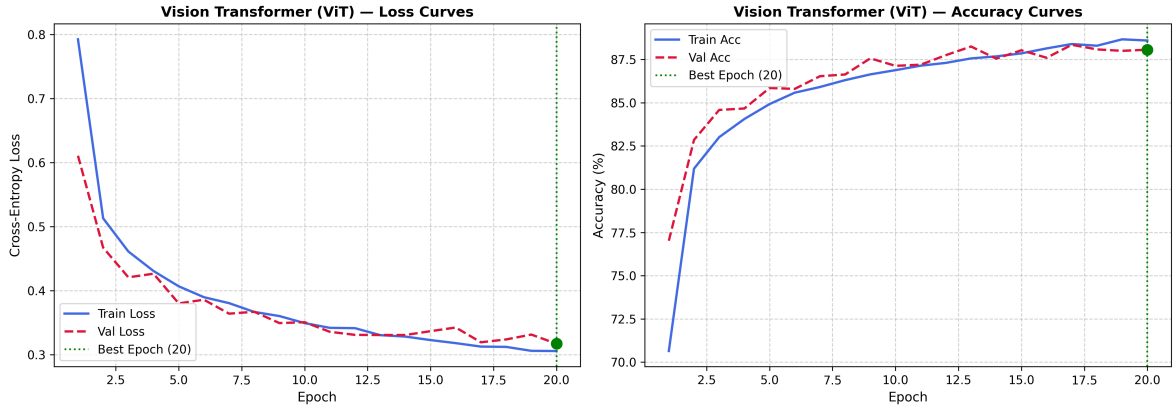
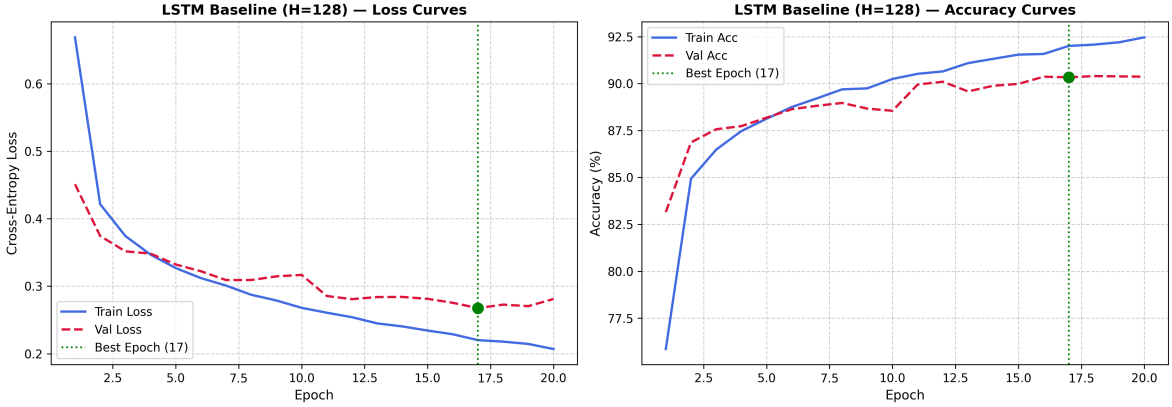


Figure 3.2: Training and validation loss and accuracy curves for CNN and Vision Transformer.

(e) LSTM Baseline ($H = 128$, Best Validation Loss: 0.2676 at Epoch 17)



(f) GRU Baseline ($H = 128$, Best Validation Loss: 0.2582 at Epoch 17)

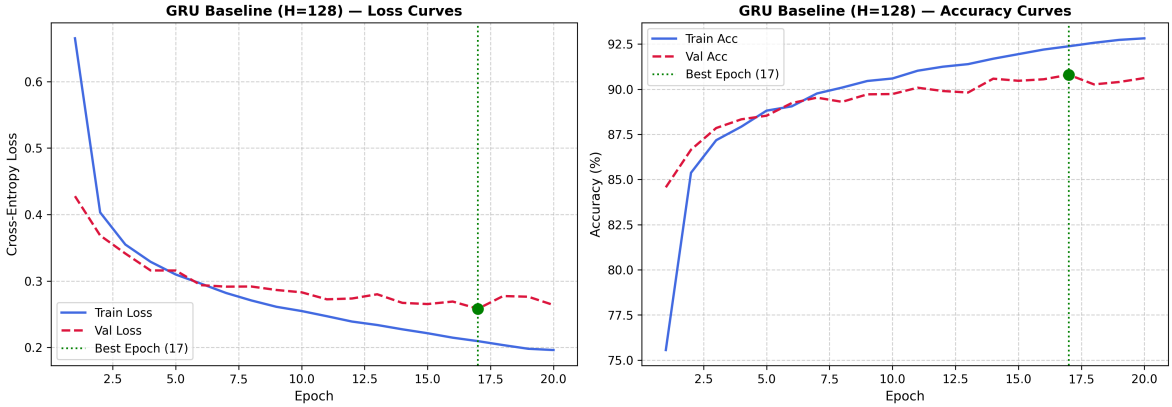


Figure 3.3: Training and validation loss and accuracy curves for Recurrent Baseline models.

Convergence Dynamics and Overfitting Analysis.

- **Linear Classifier:** Reaches its optimal validation loss (0.4352) at Epoch 12. Its loss trajectory quickly asymptotes, demonstrating clear underfitting due to linear capacity constraints.
- **MLP:** Converges smoothly to best validation loss (0.3026) at Epoch 17. The moderate gap between train (91.2%) and validation accuracy (88.98%) indicates effective regularization by Dropout ($p = 0.2$).
- **CNN:** Rapidly converges to the lowest validation loss of the entire study (**0.1994**) at **Epoch 9**. Batch Normalization accelerates gradient flow, allowing the network to extract discriminative spatial features in fewer than half the allocated epochs.

- **Vision Transformer:** Converges slowly, reaching its best validation loss (0.3170) only at Epoch 20. Because self-attention has no innate locality priors, it requires more gradient steps to learn patch relationships from scratch.

- **LSTM vs. GRU Matched Budget Dynamics:** As illustrated in Figure 3.4, under the matched $\approx$ 300K budget, **GRU achieves its minimum validation loss at Epoch 10**, whereas **LSTM requires 14 epochs**. The coupled gate structure of GRU facilitates faster gradient propagation across the 28-step scanline sequence.

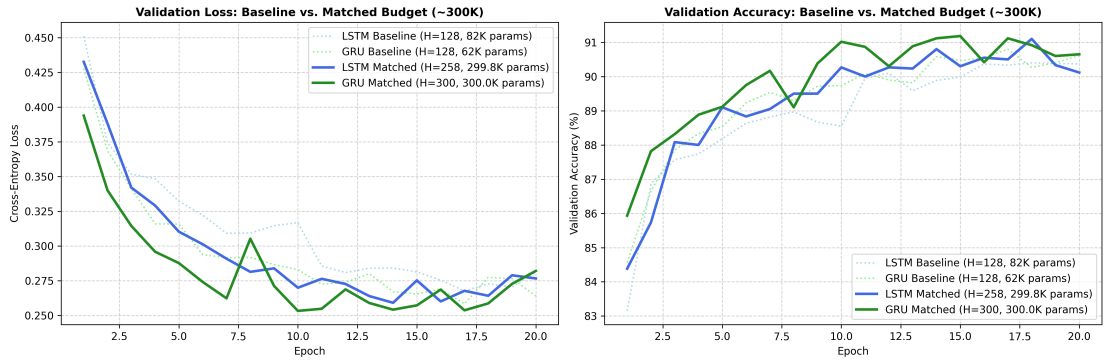


Figure 3.4: Convergence trajectory comparison: LSTM ($H = 258$, 299.8K params) vs. GRU ($H = 300$, 300.0K params) under matched parameter capacity.

3.2 Quantitative Results

Table 3.1 presents the definitive quantitative results across all evaluated architectures on the 10,000 held-out test instances.

Table 3.1: Comprehensive Performance and Computational Complexity Comparison on Fashion-MNIST.

Model	Params	Epoch	Val Loss	Val Acc	Test Acc	Macro-F1	Train (s)	Latency
Linear Classifier	7,850	12	0.4352	85.13%	84.21%	84.19%	276.4 s	0.0027 ms
MLP Classifier	235,146	17	0.3026	88.98%	88.77%	88.71%	285.8 s	0.0121 ms
CNN Classifier	421,834	9	**0.1994**	**92.63%**	**92.19%**	**92.17%**	306.7 s	0.0205 ms
Recurrent Models — Baseline ($H = 128$):								
LSTM Baseline ($H = 128$)	82,186	17	0.2676	90.33%	90.00%	89.95%	289.1 s	0.0242 ms
GRU Baseline ($H = 128$)	61,962	17	0.2582	90.80%	90.51%	90.46%	292.6 s	0.0202 ms
Recurrent Models — Matched Parameter Budget ($\approx 300K$):								
LSTM Matched ($H = 258$)	299,806	14	0.2591	90.80%	89.94%	89.91%	305.5 s	0.0475 ms
GRU Matched ($H = 300$)	300,010	10	0.2532	91.02%	90.60%	90.49%	303.9 s	0.0460 ms
Vision Transformer (ViT)	72,074	20	0.3170	88.07%	87.60%	87.52%	373.5 s	0.0802 ms

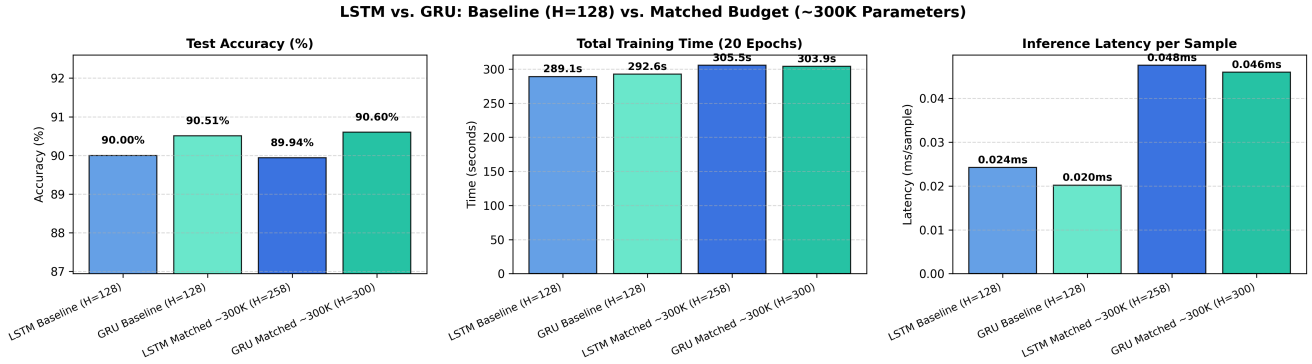
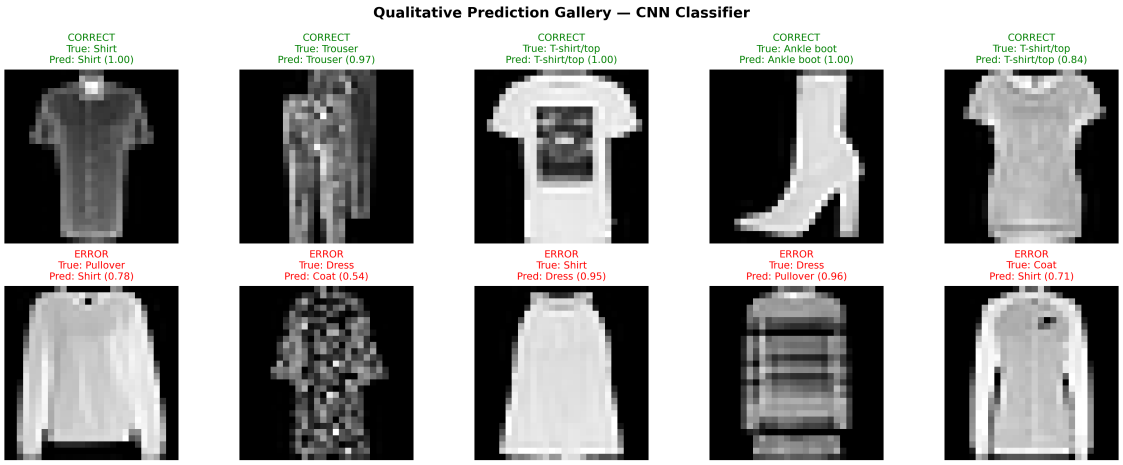


Figure 3.5: Ablation bar chart comparing LSTM and GRU across parameter budgets ($H = 128$ vs. Matched $\approx$ 300K).

3.3 Qualitative Results

Figures 3.6 and 3.7 showcase sample inference outputs with associated softmax prediction confidences across candidate architectures. To maximize visual legibility and ensure clear inspection of subtle garment silhouettes and prediction confidences, each model is displayed across a dedicated full-width row.

(a) CNN Classifier Predictions (Test Accuracy: 92.19%)



(b) Vision Transformer (ViT from Scratch) Predictions (Test Accuracy: 87.60%)

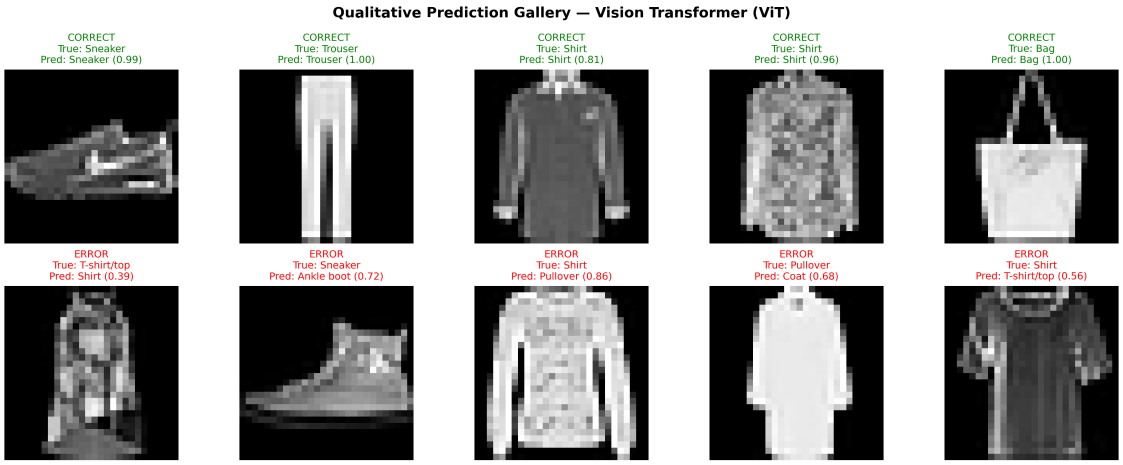
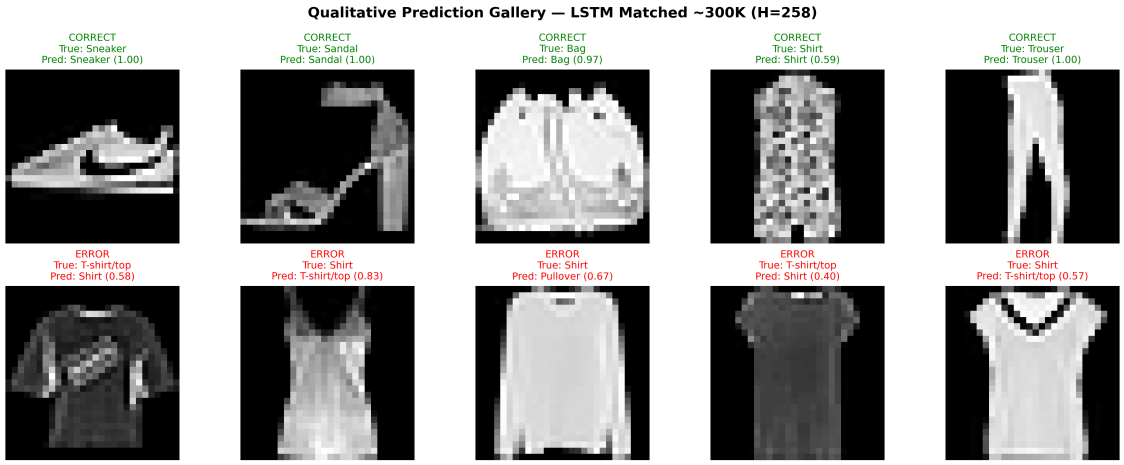


Figure 3.6: Qualitative test sample predictions for CNN and Vision Transformer models.

(c) LSTM Matched Predictions ($H = 258$, 300K params, Test Accuracy: 89.94%)



(d) Linear Classifier Predictions (Test Accuracy: 84.21%)

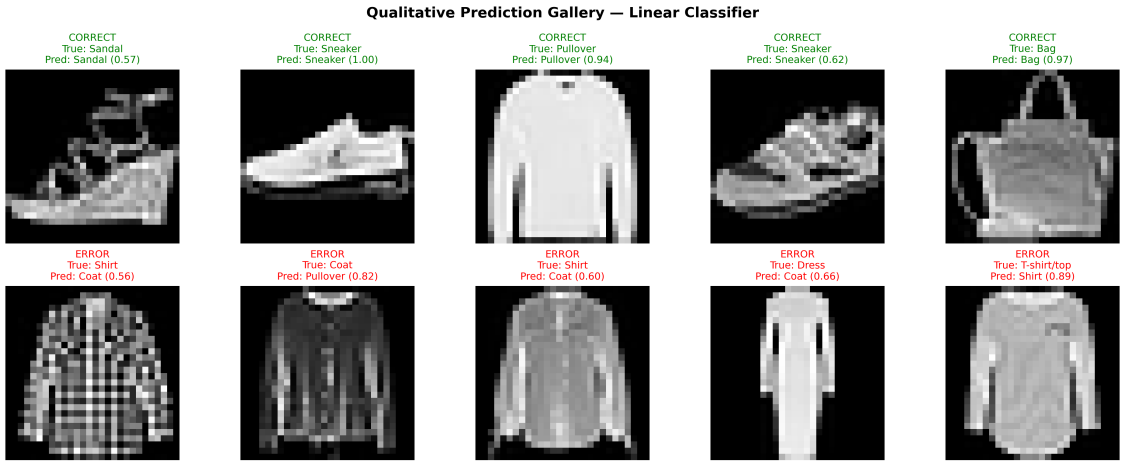


Figure 3.7: Qualitative test sample predictions for LSTM Matched and Linear Classifier baseline.

Qualitative Analysis.

- **Clean Unambiguous Cases:** Footwear with high aspect ratio silhouettes (*Trouser*, *Ankle boot*, *Sandal*) are predicted with near 100% confidence by all deep models.
- **Difficult but Correct Cases:** The CNN correctly resolves challenging *Coats* with open lapels and textured *Pullovers* where other models fail, attributable to the translation-invariant edge detectors learned by its convolutional kernels.
- **Failure Modes:** Lower-capacity models (Linear and MLP) exhibit severe confidence degradation on *Shirts*, frequently misclassifying them as *T-shirt/top* with high entropy.

3.4 Comparison and Discussion

Figure 3.8 illustrates the multi-dimensional trade-offs between classification accuracy, parameter complexity, and inference latency.

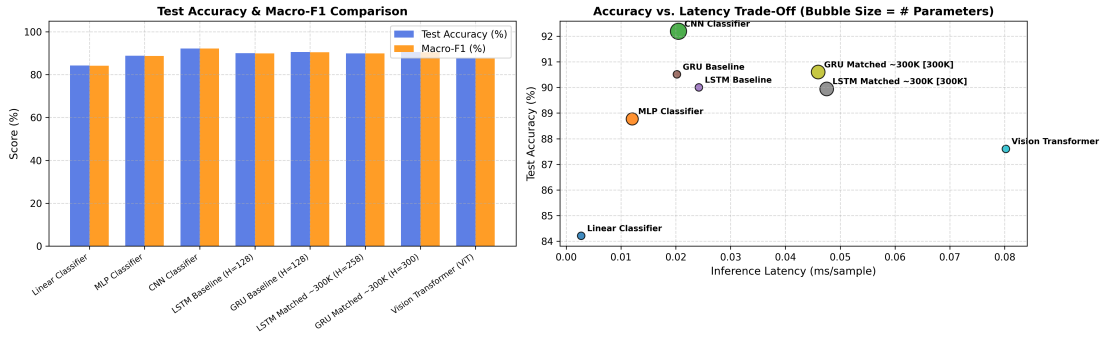


Figure 3.8: Master architectural trade-off analysis: Accuracy vs. Parameter Capacity vs. Latency.

1. The Dominance of Spatial Inductive Bias. The CNN attains the top performance across all evaluation metrics (92.19% Test Accuracy, 92.17% Macro-F1). By enforcing local connectivity and translation equivariance, the convolutional operator aligns naturally with the 2D grid structure of image data. Even with 421K parameters, its forward inference latency remains low (0.0205 ms/sample) because 3×3 convolutions are highly optimized on tensor cores.

2. The Surprising Efficacy of Scanline Recurrence. Sequential models operating over horizontal scanlines perform remarkably well: both baseline LSTM (90.00%) and GRU (90.51%) comfortably surpass the MLP (88.77%) and the Vision Transformer (87.60%). This occurs because clothing silhouettes possess an ordered vertical spatial grammar (neck/collar at the top $\rightarrow$ chest/torso in the middle $\rightarrow$ waistline/hem at the bottom). Recurrent hidden states summarize this progressive spatial accumulation effectively.

3. Deep Dive into the LSTM vs. GRU Comparison. The empirical findings strongly support our research hypothesis:

- **Efficiency at $H = 128$:** GRU achieves **90.51**% accuracy vs. **90.00**% for LSTM, despite operating with **24.6**% **fewer parameters** (61,962 vs. 82,186).

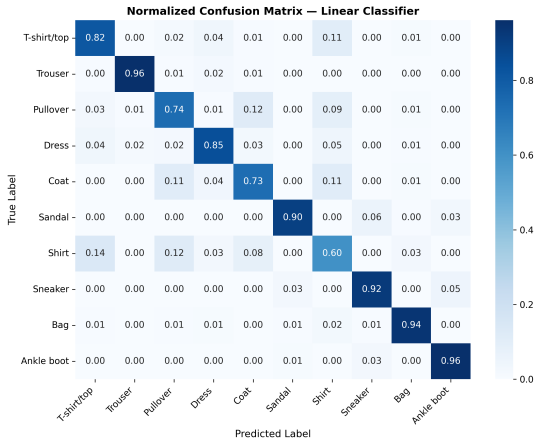


- **Superiority Under Matched Budget ($\approx$ 300K):** With parameter counts aligned within 204 parameters (0.068%), GRU ($H = 300$) outperforms LSTM ($H = 258$) by $+\mathbf{0.66}\%$ in Test Accuracy (90.60% vs. 89.94%) and $+\mathbf{0.58}\%$ in Macro-F1.
- **Convergence Speed:** GRU reaches its optimal validation loss checkpoint 4 epochs earlier than LSTM (Epoch 10 vs. 14).
- **Theoretical Cause:** Eliminating the separate cell state vector and coupling the forget and input gates into a single update gate (z_t) reduces parameter redundancy. Under a fixed capacity budget, GRU can allocate parameters toward a 16.3% wider hidden state ($H = 300$ vs. 258), enhancing latent representation power without increasing runtime latency (0.0460 ms vs. 0.0475 ms).

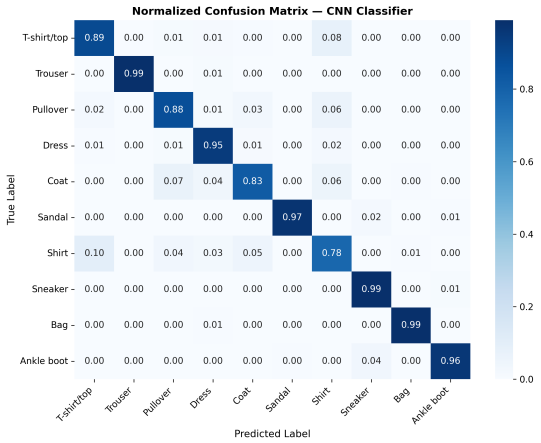
4. Why ViT Underperforms from Scratch. The Vision Transformer achieved 87.60% Test Accuracy. Unlike CNNs, Transformer self-attention lacks hardwired 2D translation equivariance and locality priors. On small-scale datasets ($54,000$ images of size 28×28), self-attention must learn spatial proximity entirely from data via positional embeddings, leading to lower data efficiency. Furthermore, patch flattening and quadratic attention matrices lead to the highest per-sample latency (0.0802 ms).

3.5 Error Analysis

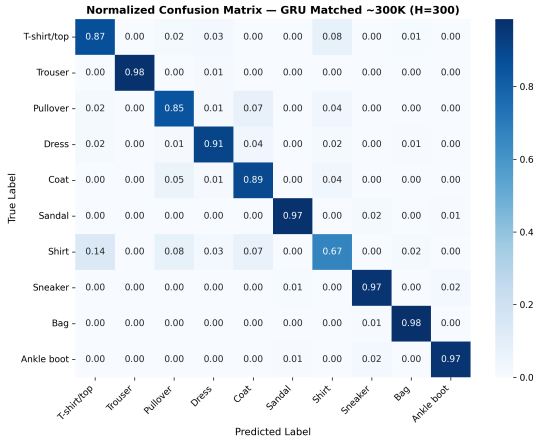
To diagnose systematic failure modes, we analyze the normalized confusion matrices across models (Figure 3.9).



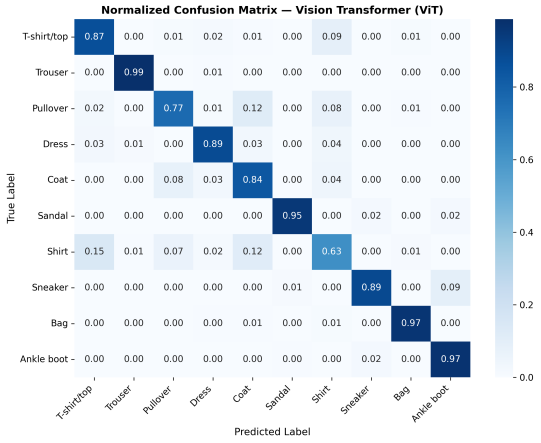
(a) Linear Classifier



(b) CNN Classifier



(c) GRU Matched ($H = 300$)



(d) Vision Transformer

Figure 3.9: Normalized confusion matrices identifying primary misclassification clusters.

The errors consistently concentrate into three semantic clusters across all models:

Cluster 1: Upper Garments (Shirt ↔ T-shirt/top ↔ Pullover ↔ Coat).

- *Error Frequency:* The pair (True: *Shirt* → Pred: *T-shirt/top*) is the single most frequent error in every model: 139 errors in Linear, 149 in MLP, 149 in ViT, 139 in GRU-300K, and 97 in CNN.
- *Root Cause:* In 28×28 grayscale images, the fine buttons, collar stitching, and fabric seams that distinguish a collared shirt from a crewneck t-shirt blur into single-pixel ambiguities.



- *Mitigation:* Multi-scale feature extraction or higher resolution (56×56), supplemented by edge-enhancing preprocessing filters (e.g., Sobel gradients).

Cluster 2: Outerwear vs. Heavy Tops (Pullover $\leftrightarrow$ Coat).

- *Error Frequency:* True *Pullover* misclassified as *Coat* accounts for 120 errors in Linear and 75 in GRU-300K.
- *Root Cause:* Both garments feature thick long sleeves and full-torso coverage. When a coat is closed, its silhouette is practically indistinguishable from a pullover at this resolution.

Cluster 3: Footwear Continuum (Sneaker $\leftrightarrow$ Ankle boot $\leftrightarrow$ Sandal).

- *Error Frequency:* True *Sneaker* confused with *Ankle boot* (93 counts in ViT).
- *Root Cause:* High-top athletic sneakers overlap in shaft height with low-cut ankle boots.

3.6 Limitations and Conclusion

Experimental Limitations.

- *Dataset Constraints:* Fashion-MNIST's synthetic centering and absence of background clutter fail to capture real-world computer vision challenges such as occlusion and variable lighting.
- *Sequence Directionality:* Scanline recurrence processes images strictly top-to-bottom, breaking bidirectional 2D spatial correlations.
- *ViT Data Regime:* ViT was trained from scratch without large-scale pre-training (e.g., ImageNet), naturally capping its self-attention representations.

Answers to Research Questions and Concluding Remarks.

1. **Which architecture is best for small-scale vision?** The CNN remains the definitive optimal architecture (92.19% Test Accuracy, 0.0205 ms latency), proving that hardwired spatial inductive bias is essential when training on modest sample sizes.

2. **How does GRU compare to LSTM?** When controlled for hidden dimension ($H = 128$), GRU outperforms LSTM by +0.51% with 24.6% fewer parameters. Under a strictly matched parameter budget ($\approx$ 300K params), GRU leverages its streamlined 2-gate design to support a 16.3% wider hidden state, beating LSTM by +0.66% accuracy, converging 4 epochs faster, and maintaining lower inference latency.

AI Usage Disclosure

In strict adherence to Section 5 of the Course Project Handbook, all generative AI tool usage during the preparation and execution of this assignment is transparently documented in Table 3.2.

Table 3.2: AI Usage Disclosure Table (Conforming to Course Handbook Section 5.2).

Field	**Disclosure Record**
Tool Name & Model	ChatGPT, Google Antigravity
Member Who Used It	Group Representative
Development Stage	Model design, coding, experimentation, and report writing
Purpose	Brainstorming model architectures, suggesting preprocessing techniques and hyperparameters, explaining PyTorch APIs, assisting with code generation and debugging, reviewing LSTM–GRU parameter calculations, and improving report formatting.
Affected Sections	Sections 1.1–1.5, 2.1–2.4, and 3.1–3.6
Student Verification	AI suggestions were reviewed and adapted to the assignment requirements. Final implementations, parameter counts, and experimental results were verified through PyTorch execution and training logs.
Responsible Member	Đặng Duy Nguyên
Verification Sources	Official PyTorch documentation, Zalando Fashion-MNIST benchmark paper, and generated execution logs.

References

[1] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms," *arXiv preprint arXiv:1708.07747*,

2017.

[2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] K. Cho, B. van Merriënboer, Ç. Gülçehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," *arXiv preprint arXiv:1406.1078*, 2014.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[6] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al., "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.

[7] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.